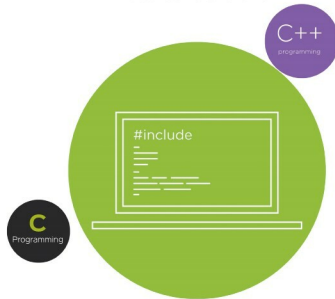


The Minor Differences Between C and C++

This article discusses the subtle and minute differences between C and C++.



An article titled ‘Major Differences Between C and C++’ would have been quite ridiculous because everybody knows that C and C++ are two independent programming languages, both backed by communities that are quite fanatical in their support for their respective preference. There will probably be a small group of novice programmers who still post questions in StackOverflow with the tag ‘C/ C++’ and will get panned universally by professional programmers belonging to both the camps. In this article, I plan to point out a few subtle differences between C and C++, altogether avoiding the major differences like C being an imperative programming language whereas C++ is an object-oriented programming language. This article tries to point out that even though text books often define C++ as a superset of C, even that part of C++ that is a subset of C is slightly different from traditional C.

I will try to prove the existence of such minute differences between C and C++ that often escape the notice of even seasoned programmers with a small example. This is a question I have asked recently in a programming competition: “Write an error-free C program that will give an error when compiled as a C++ program.” Since the classical definition of C++ says that it is a superset of C, every C program should also compile as a C++ program without any errors. This might be the case theoretically though not so practically. The following program *Example 1* fails when compiled with the g++ compiler as a C++ program, but when compiled as a C

program with the gcc compiler, it compiles successfully.

```
#include<stdio.h>
int main()
{
    int class;
}
```

Example 1

What is the reason for the error when compiled as a C++ program? That’s very simple—the keyword `class` in C++ can be used as a variable name in C. But to crown it all, the first prize went to a programming whizz-kid who knew that anonymous structures are okay in C but will lead to an error in C++. The program *Example 2* with an anonymous structure will compile as a C program with a warning, whereas in C++ the program will result in an error. All the programs in this article are compiled as C programs with gcc and as C++ programs with g++.

```
#include<stdio.h>

struct
{
    int a;
};
```